

Dockerfile Avanzado

Arturo Silvelo

Try New Roads

Introducción a Docker Compose avanzado

Docker Compose es una herramienta esencial para definir y gestionar aplicaciones multicontenedor en Docker. Permite describir la arquitectura de una aplicación, sus servicios, redes y volúmenes en un solo archivo YAML, facilitando la orquestación y el despliegue en diferentes entornos.

Fragments

En archivos YAML, como los usados por Docker Compose, los anchors (`&`) y alias (`*`) permiten reutilizar bloques de configuración, evitando duplicidad y facilitando el mantenimiento.

- **Anchor** (`&`): Define un bloque reutilizable.
- **Alias** (`*`): Inserta el bloque definido por el anchor.
- **Merge** (`<<`): Permite combinar configuraciones.

Ventajas

- Menos repetición de código.
- Cambios centralizados y más fáciles de mantener.
- Configuraciones más limpias y legibles.

```
services:
  base-1:
    image: app-base
    container_name: base-1
    ports:
      - "3000:5000"
    environment: &env
      - PORT=5000
      - LOG_LEVEL=debug

  base-2:
    image: app-base
    container_name: base-2
    ports:
      - "4000:5000"
    environment: *env

  base-3:
    image: app-base
    container_name: base-3
    ports:
      - "5000:8000"
    environment: &env-list
      PORT: 8000
      LOG_LEVEL: warn

  base-4:
    image: app-base
    container_name: base-4
    ports:
      - "6000:8000"
    environment:
      <<: *env-list
      LOG_LEVEL: info
```

- Ejecución

```
docker compose -f 1.anchor/docker-compose.yaml up
```

- Verificación

```
docker exec base-1
docker exec base-2
docker exec base-3
docker exec base-4
```

- Resultado

```
LOG_LEVEL=debug
PORT=5000
```

- Limpieza

```
docker compose -f 1.anchor/docker-compose.yaml down
```

Extensions

Las extensiones (`x-`) permiten crear configuraciones modulares y reutilizables en Docker Compose. Son campos personalizados que Compose ignora, pero que puedes usar con anchors y aliases para hacer tus archivos más eficientes y mantenibles.

- **Prefijo `x-`**: Define configuraciones modulares reutilizables
- **Ignorados por Compose**: Solo sirven para organización y reutilización
- **Combinables con anchors**: Máxima flexibilidad de configuración

Ventajas de Extensions

- **Modularidad:** Separa configuraciones complejas en bloques reutilizables
- **Mantenibilidad:** Cambios centralizados en un solo lugar
- **Legibilidad:** Archivos Compose más limpios y organizados
- **Experimentación:** Soporte para features no oficiales

```
x-base: &base
  image: app-base
  container_name: base-1
  ports:
    - "3000:5000"
  environment:
    - PORT=5000
    - LOG_LEVEL=debug

services:
  base-1:
    <<: *base
  base-2:
    <<: *base
    container_name: base-2
    ports:
      - "4000:5000"

  base-3:
    <<: *base
    container_name: base-3
    ports:
      - "5000:5000"

  base-4:
    <<: *base
    container_name: base-4
    ports:
      - "6000:5000"
    environment:
      LOG_LEVEL: info

networks:
  default:
    driver: bridge
    name: app-base
```

• Ejecución

```
docker compose -f 02-compose/ejemplos/2.extension/docker-compose.yaml up -d
```

• Verificación

```
docker compose -f 02-compose/ejemplos/2.extension/docker-compose.yaml ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
69fc8a8e740f	app-base	"docker-entrypoint.s..."	6 seconds ago	Up 6 seconds (healthy)	3000/tcp, 0.0.0.0:4000->5000/tcp, [::]:4000->5000/tcp	base-2
4c45af3a3ce2	app-base	"docker-entrypoint.s..."	6 seconds ago	Up 6 seconds (healthy)	3000/tcp, 0.0.0.0:3000->5000/tcp, [::]:3000->5000/tcp	base-1
ec7edaa58407	app-base	"docker-entrypoint.s..."	6 seconds ago	Up 6 seconds (healthy)	3000/tcp, 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	base-3
54624a094099	app-base	"docker-entrypoint.s..."	6 seconds ago	Up 6 seconds (healthy)	3000/tcp, 0.0.0.0:6000->5000/tcp, [::]:6000->5000/tcp	base-4

• Limpiar

```
docker compose -f 02-compose/ejemplos/2.extension/docker-compose.yaml down
```


Healthcheck

El parámetro `healthcheck` en Docker Compose permite definir una comprobación periódica para saber si un servicio está funcionando correctamente. Esto ayuda a detectar fallos y a gestionar dependencias entre servicios.

El parámetro `healthcheck` puede definirse tanto en el `Dockerfile` como en cada servicio dentro de Docker Compose.

En ambos casos, se dispone de varias opciones de configuración:

- **test**: Comando que se ejecuta para comprobar la salud.
- **interval**: Frecuencia de la comprobación.
- **timeout**: Tiempo máximo de espera para la comprobación.
- **retries**: Número de intentos antes de marcar el servicio como unhealthy.
- **start_period**: Tiempo de gracia antes de empezar a comprobar.

`depends_on`

Es posible emplear `depends_on` junto con la condición `service_healthy` para garantizar que un servicio espere a que otro esté en estado saludable antes de iniciar su ejecución.

El servicio dependiente será creado y su contenedor iniciado, pero no comenzará su proceso principal hasta que el servicio del que depende alcance el estado saludable (`healthy`).

```
services:
  base-health:
    container_name: base-health
    image: app-base
    environment: &env-base
    USE_DB: true
    DB_HOST: postgres
    DB_PORT: 5432
    DB_USER: postgres
    DB_PASS: password
    DB_NAME: postgres
    ports:
      - "3000:3000"

  base-no-health:
    container_name: base-no-health
    image: app-base
    ports:
      - "4000:3000"
    environment:
      <<: *env-base
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 5s
      timeout: 10s
      retries: 3
    depends_on:
      postgres:
        condition: service_healthy

  base-health-conditional:
    container_name: base-health-conditional
    image: app-base
    depends_on:
      base-no-health:
        condition: service_healthy

  base-conditional:
    container_name: base-conditional
    image: app-base
    depends_on:
      - base-no-health

  postgres:
    container_name: postgres
    image: postgres:15
    environment: &db-env
    POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3
```

• Ejecución

```
docker compose -f 2.healthcheck/docker-compose.yaml up -d
```

• Verificación

```
docker compose -f 2.healthcheck/docker-compose.yaml ps -a
```

• Resultado

```
docker compose -f 02-compose/ejemplos/3.healthcheck/docker-compose.yaml ps -a
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
base-conditional	app-base	"docker-entrypoint.s..."	base-conditional	40 seconds ago	Up 18 seconds (healthy)	3000/tcp
base-health	app-base	"docker-entrypoint.s..."	base-health	40 seconds ago	Exited (1) 39 seconds ago	
base-health-conditional	app-base	"docker-entrypoint.s..."	base-health-conditional	40 seconds ago	Created	
base-no-health	app-base	"docker-entrypoint.s..."	base-no-health	40 seconds ago	Up 19 seconds (unhealthy)	0.0.0.0:4000->3000/tcp, [::]:4000->3000/tcp
postgres	postgres:15	"docker-entrypoint.s..."	postgres	40 seconds ago	Up 39 seconds (healthy)	5432/tcp

• Limpieza

```
docker compose -f 2.healthcheck/docker-compose.yaml down
```

Gestión entornos

Multiple Ficheros `.env`

Docker Compose permite trabajar con múltiples archivos de variables de entorno para gestionar configuraciones específicas por entorno. Esto facilita la separación de configuraciones sensibles y la personalización según el contexto de ejecución.

- Fichero `.env`

```
TAG=latest
INTERNAL_PORT=3000
EXTERNAL_PORT=3000
```

- Fichero `.env.dev`

```
SECRET=development-secret
DEBUG=info
```

- Fichero `.env.prod`

```
SECRET=production-secret
DEBUG=warn
```

- Fichero `compose.yaml`

```
services:
  base:
    image: app-base:${TAG}
    container_name: app-base
    ports:
      - ${EXTERNAL_PORT}:${INTERNAL_PORT}
```

- Fichero `compose.dev.yaml`

```
services:
  base:
    image: app-base:${TAG}
    container_name: app-base
    ports:
      - ${EXTERNAL_PORT}:${INTERNAL_PORT}
    env_file:
      - .env.dev
  ...
```

- Fichero `compose.prod.yaml`

```
services:
  base:
    image: app-base:${TAG}
    container_name: app-base
    ports:
      - ${EXTERNAL_PORT}:${INTERNAL_PORT}
    env_file:
      - .env.prod
```

- Ejecución

```
docker compose --env-file 02-compose/ejemplos/9.env/.env --env-file 02-compose/ejemplos/9.env/.env.dev -f 02-compose/ejemplos/9.env/docker-compose.yaml config
```

```
docker compose -f 02-compose/ejemplos/9.env/docker-compose.dev.yaml config
```

Multiple Ficheros Compose

Docker Compose permite trabajar con múltiples archivos para personalizar aplicaciones según diferentes entornos o flujos de trabajo. Esto es especialmente útil para aplicaciones grandes con múltiples equipos y configuraciones complejas.

Ventajas:

- **Modularidad:** Separación por equipos o funcionalidades
- **Entornos:** Configuraciones específicas (dev, test, prod)
- **Reutilización:** Composición flexible de servicios
- **Mantenibilidad:** Gestión distribuida de configuraciones

Estrategias con múltiples archivos

Merge - Fusión de archivos

Docker Compose puede combinar múltiples archivos usando el flag `-f` o la variable de entorno `COMPOSE_FILE`. Los archivos se fusionan en el orden especificado, donde los archivos posteriores pueden sobrescribir, fusionar o añadir configuraciones a los anteriores.

```
docker compose -f compose.yaml -f compose.admin.yaml run backup_db
```

- `docker-compose.yaml`

```
services:
  app-base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
    ports:
      - "3000:3000"
    image: app-base
    container_name: app-base
    environment:
      NODE_ENV: development
      PORT: 3000

networks:
  default:
    driver: bridge
    name: app-base
```

- `docker-compose.dev.yaml`

```
services:
  app-base:
    environment:
      SECRET: development-secret
      DEBUG_LEVEL: debug
      USE_DB: false
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/5.merge/docker-compose.yaml -f 02-compose/ejemplos/5.merge/docker-compose.dev.yaml up --build
```

- Verificación

```
docker exec app-base env
```

```
curl http://localhost:3000/secret
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/5.merge/docker-compose.yaml -f 02-compose/ejemplos/5.merge/docker-compose.dev.yaml down
```

- `docker-compose.prod.yaml`

```
services:
  app-base:
    volumes:
      - app-logs:/app/logs
      - app-uploads:/app/uploads
    environment:
      SECRET: production-secret
      DEBUG_LEVEL: warn
      USE_DB: true
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASS: password
      DB_NAME: postgres
    depends_on:
      postgres:
        condition: service_healthy
  postgres:
    container_name: postgres
    image: postgres:15
    environment: &db-env
    POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3
volumes:
  app-logs:
    driver: local
  app-uploads:
    driver: local
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/5.merge/docker-compose.yaml -f 02-compose/ejemplos/5.merge/docker-compose.prod.yaml up --build
```

- Verificación

```
docker exec app-base env
```

```
curl http://localhost:3000/secret
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/5.merge/docker-compose.yaml -f 02-compose/ejemplos/5.merge/docker-compose.prod.yaml down
```

Extensión de archivos

Extends permite que un servicio herede configuración de otro servicio definido en un archivo diferente, seleccionando específicamente qué partes usar y permitiendo sobrescribir atributos según las necesidades.

- common-services.yaml

```
services:
  common-base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
    ports:
      - "3000:3000"
    image: app-base
    container_name: app-base
    environment:
      NODE_ENV: development
      PORT: 3000

  common-postgres:
    container_name: postgres
    image: postgres:15
    environment: &db-env
      POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/6.extends/docker-compose.dev.yaml up -d --build
```

- docker-compose.dev.yaml

```
services:
  app-base:
    extends:
      file: common-services.yaml
      service: common-base
    environment:
      SECRET: development-secret
      DEBUG_LEVEL: debug
      USE_DB: false
```

- Verificación

```
docker exec app-base env
```

```
curl http://localhost:3000/secret
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/6.extends/docker-compose.dev.yaml down
```

- `docker-compose.prod.yaml`

```
services:
  app-base:
    extends:
      file: common-services.yaml
      service: common-base
    volumes:
      - app-logs:/app/logs
      - app-uploads:/app/uploads
    environment:
      SECRET: production-secret
      DEBUG_LEVEL: warn
      USE_DB: true
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASS: password
      DB_NAME: postgres
    depends_on:
      postgres:
        condition: service_healthy

  postgres:
    extends:
      file: common-services.yaml
      service: common-postgres

volumes:
  app-logs:
    driver: local
  app-uploads:
    driver: local
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/6.extends/docker-compose.dev.yaml up -d --build
```

- Verificación

```
docker exec app-base env
```

```
curl http://localhost:3000/secret
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/6.extends/docker-compose.dev.yaml down
```


Include

Include permite incorporar archivos Compose separados directamente en tu archivo principal. Esto facilita la modularización de aplicaciones complejas en sub-archivos Compose, haciendo las configuraciones más simples y explícitas.

- app-base.yaml

```
services:
  app-base:
    image: app-base
    container_name: app-base
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
    ports:
      - "3000:3000"

    environment:
      NODE_ENV: development
      PORT: 3000
```

- postgres.yaml

```
services:
  postgres:
    container_name: postgres
    image: postgres:15
    environment: &db-env
      POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3
```

- docker-compose.override.dev.yaml

```
services:
  app-base:
    environment:
      SECRET: development-secret
      DEBUG_LEVEL: debug
      USE_DB: false
```

- docker-compose.dev.yaml

```
include:
  - path:
      - app-base.yaml
      - docker-compose.override.dev.yaml
```

- docker-compose.prod.yaml

```
include:
  - path:
      - app-base.yaml
      - postgres.yaml
      - docker-compose.override.prod.yaml

volumes:
  app-logs:
    driver: local
  app-uploads:
    driver: local
```

- `docker-compose.override.prod.yaml`

```
services:
  app-base:
    volumes:
      - app-logs:/app/logs
      - app-uploads:/app/uploads
    environment:
      SECRET: production-secret
      DEBUG_LEVEL: warn
      USE_DB: true
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASS: password
      DB_NAME: postgres
    depends_on:
      postgres:
        condition: service_healthy
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/7.include/docker-compose.dev.yaml up -d --build
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/7.include/docker-compose.dev.yaml down
```

- Verificación

```
docker exec app-base env
```

```
curl http://localhost:3000/secret
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/7.include/docker-compose.prod.yaml up -d --build
```

- Limpiar

```
docker compose -f 02-compose/ejemplos/7.include/docker-compose.dev.yaml down
```

Compose Specifications

Build Specification

La especificación Build define configuraciones avanzadas para construir imágenes Docker desde código fuente, incluyendo argumentos, targets multi-stage, cache, y builds multi-plataforma.

```
services:
  base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
      args:
        NODE: 20
      target: build
      platforms:
        - linux/amd64
        - linux/arm64
      tags:
        - "app-base:1"
        - "app-base:latest"
    ports:
      - "3000:3000"
```

Deploy Specification

La especificación Deploy define configuraciones para despliegue en entornos de producción, especialmente útil para Docker Swarm, incluyendo réplicas, recursos, actualizaciones y políticas de reinicio.

```
services:
  base:
    image: app-base:1
    deploy:
      mode: replicated
      replicas: 2
      resources:
        limits:
          cpus: "1"
          memory: 512M
        reservations:
          cpus: "0.5"
          memory: 256M
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
```


Develop Specification

La especificación Develop define configuraciones específicas para desarrollo local, incluyendo hot reload, file watching, debugging, y sincronización de archivos en tiempo real.

- `docker-compose.develop.yaml`

```
services:
  base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
      target: test
    environment:
      - USE_DB=false
    ports:
      - "3000:3000"
    develop:
      watch:
        - action: sync+restart
          path: ../../../../app/src
          target: /app/src
          ignore:
            - node_modules/
            - "**/*.test.js"
        - action: rebuild
          path: ../../../../app/package.json
    volumes:
      - node_modules:/app/node_modules
    command: ["npm", "start"]

volumes:
  node_modules:
```

- Ejecución

```
docker compose -f 02-compose/ejemplos/8.build/docker-compose.develop.yaml watch
```

- Verificación

- Modificar el código hará que reinicie el servidor
- Modificar el package.json hará un nuevo `build`

- Limpiar

```
docker compose -f 02-compose/ejemplos/8.build/docker-compose.develop.yaml down
```

Resumen

Specification	Fase	Propósito Principal	Comando Típico	Entorno
Develop	Desarrollo	Hot reload, debugging	<code>docker compose watch</code>	Local
Build	Construcción	Imágenes optimizadas	<code>docker compose build</code>	CI/CD
Deploy	Producción	Orquestación escalable	<code>docker stack deploy</code>	Swarm/Producción